Assignment 8 - Big Data Analytics with Docker container

Objective: In this assignment, you will build a complete local Big Data Analytics platform using Docker instead of AWS EMR. This assignment works on: Windows, macOS Intel, macOS Apple Silicon (M1/M2/M3). You will perform the following task listed below:

- Install Docker
- Build Hadoop + Spark Cluster
- Run Spark Analytics Jobs
- Perform SQL Analytics
- Use Kafka Streaming
- Build Dashboards using Superset
- Use Jupyter Notebook for Analytics
- Store Data in PostgreSQL

AWS Service Vs Docker

AWS Service	Docker/Open Source Replacement
S3	Local mounted volume / MinIO
AWS Glue	PySpark ETL jobs
Athena	Spark SQL / Hive
Kinesis	Apache Kafka
EMR	Spark + Hadoop cluster
QuickSight	Apache Superset
Glue Data Catalog	Hive Metastore

TECHNOLOGIES USED

Technology	Purpose
Docker	Containerization
Hadoop HDFS	Distributed Storage
Apache Spark	Big Data Processing

PostgreSQL	Metadata & Warehouse
Kafka	Real-Time Streaming
Jupyter Notebook	Analytics Development
Apache Superset	Dashboard Visualization

Assignment Architecture

CSV Dataset
↓
Spark ETL Processing
↓
Cleaned Data
↓
Spark SQL Analytics
↓
PostgreSQL Warehouse
↓
Superset Dashboards

Deliverables: Each student must submit the following:

Presentation Slides (20 points)
- Title slide with student name(s), course, and assignment title.
- Give a clear explanation of each completed task.
- Screenshots or videos of key steps/results.
- Lessons learned and challenges encountered.

Presentation Guidelines
- Use professional formatting (legible fonts, consistent colors, organized layout).
- Ensure clarity: avoid overcrowding slides with text; use bullet points and visuals.
- Provide screenshots or videos as evidence of work completed.
- Keep presentations within the time limit (8–11 minutes).
- Be prepared to answer questions about the process and challenges.

Professionalism & Delivery (10 points)
- Confident and clear presentation delivery
- Ability to answer questions from peers/instructor
- Team collaboration and equal participation

Submission Instructions
- Submit slides (PowerPoint or PDF) to Moodle by the deadline.

Task completion (70 points)

SECTION A — WINDOWS USER SETUP GUIDE

TASK 1 — Install Docker Desktop on Windows

- Step 1.1 — Download Docker Desktop

Open: https://www.docker.com/products/docker-desktop/

Download: Docker Desktop for Windows

- Step 1.2 — Install Docker Desktop

During installation:

- Enable WSL2 integration
- Enable Hyper-V if asked
- Restart the system after installation.
- Step 1.3 — Verify Docker Installation

Open PowerShell:

- Run: docker --version
- Expected output:

Docker version xx.xx.x

Run: docker compose version

TASK 2 — Create Project Directory

Create project folder:
mkdir bigdata-docker
cd bigdata-docker

Create folders:
mkdir data
mkdir data\source
mkdir data\target
mkdir notebooks
mkdir spark
mkdir spark\jobs
mkdir jars

TASK 3 — Create Docker Compose File

Create file: docker-compose.yml

Paste the following code:

```
version: '3.9'

services:

  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
```

```
    environment:
      CLUSTER_NAME: test
    ports:
      - "9870:9870"
      - "9000:9000"
    volumes:
      - namenode:/hadoop/dfs/name

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    environment:
      SERVICE_PRECONDITION: namenode:9870
    ports:
      - "9864:9864"
    volumes:
      - datanode:/hadoop/dfs/data
    depends_on:
      - namenode

  spark-master:
    image: apache/spark:3.5.1
    platform: linux/amd64
    container_name: spark-master
    command: >
      /opt/spark/bin/spark-class
      org.apache.spark.deploy.master.Master
    ports:
      - "8080:8080"
      - "7077:7077"
    volumes:
      - ./data:/opt/spark-data
      - ./spark/jobs:/opt/spark/jobs
      - ./jars/postgresql.jar:/opt/spark/jars/postgresql.jar
    environment:
      SPARK_EXTRA_CLASSPATH: /opt/spark/jars/postgresql.jar

  spark-worker:
    image: apache/spark:3.5.1
    platform: linux/amd64
    container_name: spark-worker
    command: >
      /opt/spark/bin/spark-class
      org.apache.spark.deploy.worker.Worker
```

```
    spark://spark-master:7077
  depends_on:
    - spark-master
  volumes:
    - ./data:/opt/spark-data
    - ./spark/jobs:/opt/spark/jobs
    - ./jars/postgresql.jar:/opt/spark/jars/postgresql.jar
  environment:
    SPARK_EXTRA_CLASSPATH: /opt/spark/jars/postgresql.jar

postgres:
  image: postgres:14
  container_name: postgres
  environment:
    POSTGRES_USER: hive
    POSTGRES_PASSWORD: hive
    POSTGRES_DB: metastore
  ports:
    - "5433:5432"

jupyter:
  image: jupyter/pyspark-notebook
  container_name: jupyter
  ports:
    - "8888:8888"
  volumes:
    - ./notebooks:/home/jovyan/work
  environment:
    JUPYTER_TOKEN: mypassword

superset:
  image: apache/superset:latest
  platform: linux/amd64
  container_name: superset
  ports:
    - "8088:8088"
  environment:
    SUPERSET_SECRET_KEY: supersecretkey123
    SUPERSET_LOAD_EXAMPLES: "yes"
  command: >
    /bin/sh -c "
    pip install psycopg2-binary &&
    superset db upgrade &&
    superset fab create-admin
```

```
      --username admin
      --firstname Admin
      --lastname User
      --email admin@example.com
      --password admin123 || true &&
    superset init &&
    superset run -h 0.0.0.0 -p 8088
    "

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

volumes:
  namenode:
  datanode:
```

TASK 4 — Download PostgreSQL JDBC Driver

Open PowerShell:

Run:

```
Invoke-WebRequest `
-Uri "https://jdbc.postgresql.org/download/postgresql-42.7.3.jar" `
-OutFile "jars/postgresql.jar"
```

TASK 5 — Start Docker Services

Run: docker compose up -d

TASK 6 — Verify Running Containers

Run: docker ps

You should see:

- namenode
- datanode
- spark-master
- spark-worker
- postgres
- jupyter
- superset
- kafka
- zookeeper

TASK 7 — Open Big Data Services

Service	URL
Hadoop HDFS	http://localhost:9870
Spark UI	http://localhost:8080
Jupyter Notebook	http://localhost:8888
Superset	http://localhost:8088

TASK 8 — Login Credentials

Jupyter Notebook

Password/Token: mypassword

Superset

Username: admin

Password: admin123

TASK 9 — Download Dataset

Open: https://www.kaggle.com/datasets/chicago/chicago-food-inspections

Download: food-inspections.csv

Rename to: **food_inspection_data.csv**

Move file into: **data/source/**

- ## Verify Dataset

Run: Get-Content data/source/food_inspection_data.csv -TotalCount 5
You should see CSV columns.

TASK 10 — Create Spark Analytics Script

Create file: spark/jobs/process.py
Paste the following code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("FoodInspectionAnalytics") \
    .getOrCreate()

print("Reading CSV file...")

# Read CSV
df = spark.read.csv(
    "/opt/spark-data/source/food_inspection_data.csv",
    header=True,
    inferSchema=True
)

print("CSV loaded successfully")

# Data Cleaning
clean_df = df.dropna()

clean_df = clean_df.filter(
    col("DBA Name").isNotNull()
)

clean_df.createOrReplaceTempView("violations")

# Analytics Queries
print("Top restaurants with most inspections")

top_restaurants = spark.sql("""
SELECT
```

```
    `DBA Name` AS restaurant_name,
    COUNT(*) AS total_inspections
FROM violations
GROUP BY `DBA Name`
ORDER BY total_inspections DESC
LIMIT 10
""")

top_restaurants.show()

print("Inspection risk distribution")

risk_distribution = spark.sql("""
SELECT
    Risk,
    COUNT(*) AS total
FROM violations
GROUP BY Risk
ORDER BY total DESC
""")

risk_distribution.show()
print("Top failure results")
failure_results = spark.sql("""
SELECT
    Results,
    COUNT(*) AS total
FROM violations
GROUP BY Results
ORDER BY total DESC
""")
failure_results.show()

# Save CSV Output

top_restaurants.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("/opt/spark-data/target/output")

# Save Parquet Output

clean_df.write.mode("overwrite").parquet(
    "/opt/spark-data/target/parquet"
```

```
)

# Save to PostgreSQL

print("Writing results to PostgreSQL...")

top_restaurants.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://postgres:5432/metastore") \
    .option("dbtable", "top_restaurants") \
    .option("user", "hive") \
    .option("password", "hive") \
    .option("driver", "org.postgresql.Driver") \
    .mode("overwrite") \
    .save()

print("PostgreSQL write complete")

spark.stop()
```

- ## Run Spark Analytics Job

Run: **docker exec -it spark-master /opt/spark/bin/spark-submit /opt/spark/jobs/process.py**

- ## Verify Output Files

Run: **dir data\target\output**

You should see: **_SUCCESS part-00000...**

- ## View Analytics Results

Run: **Get-Content data/target/output/part-*.csv**

TASK 11 — Verify PostgreSQL Data

Run: **docker exec -it postgres psql -U hive -d metastore**

Inside PostgreSQL: SELECT * FROM top_restaurants;

For exit enter the command : **\q**

TASK 12- Connect PostgreSQL to Superset

Open: **http://localhost:8088**

Go to: Settings → Database Connections → + Database

Use: **postgresql://hive:hive@postgres:5432/metastore**

If there is a connection error then run the following command.

- docker exec -u root -it superset. sh -lc "/app/.venv/bin/python -m ensurepip --upgrade"
- docker exec -u root -it superset sh -lc "/app/.venv/bin/python -m pip install psycopg2-binary"
- docker exec -it superset /app/.venv/bin/python -c "import psycopg2; print('psycopg2 OK')"
- docker restart superset

Test Connection and Save.

TASK 13 — Create Superset Dashboard

- Create: Dataset, Charts , Dashboard
- Suggested dashboard name: **Food Inspection Analytics Dashboard**

Suggested charts:
- Top Restaurants
- Risk Distribution
- Inspection Failures

TASK 14 — Kafka Streaming Analytics

- ## Create Kafka Topic

Run: **docker exec kafka kafka-topics --create --topic inspections --bootstrap-server localhost:9092**

- ## Start Kafka Producer

Run: **docker exec -it kafka kafka-console-producer --topic inspections --bootstrap-server localhost:9092**

Send sample events:
Restaurant A,Risk 1,Fail
Restaurant B,Risk 2,Pass

- ## Start Kafka Consumer

Open another terminal:

Run: **docker exec -it kafka kafka-console-consumer --topic inspections --from-beginning --bootstrap-server localhost:9092**

You should see live streaming events.

TASK 15 — Create Spark Streaming Job

Create file: spark/jobs/streaming.py

Paste the following code

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("KafkaStreaming") \
    .getOrCreate()

stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "inspections") \
    .load()

query = stream_df.writeStream \
    .format("console") \
    .start()

query.awaitTermination()
```

TASK 16 — Run Spark Streaming Job

Run: **docker exec -it spark-master /opt/spark/bin/spark-submit /opt/spark/jobs/streaming.py**

You should observe real-time Kafka data processing.

TASK 17 — Jupyter Notebook Analytics

Open: http://localhost:8888

Login using: mypassword

Create notebook: analytics.ipynb

Paste the following code.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
print("Spark Session Started")
```

Run notebook cell.

- ## Stop Docker Services

Run: docker compose down

Remove Docker Volumes

Run: **docker compose down -v**

SECTION B — macOS USER GUIDE

The steps for macOS are identical except for installation and terminal commands.

TASK 1 — Install Docker Desktop on macOS

Download: https://www.docker.com/products/docker-desktop/

Choose: Mac Intel OR Mac Apple Silicon
Install Docker Desktop.
Start Docker Desktop.

Verify Docker Installation
Open Terminal:
Run: **docker --version**
Run: **docker compose version**

TASK 2 — Create Project Directory

Run: mkdir bigdata-docker

cd bigdata-docker
Create folders:

mkdir -p data/source
mkdir -p data/target
mkdir -p notebooks
mkdir -p spark/jobs
mkdir -p jars

TASK 3 — Create Docker Compose File

Create: **docker-compose.yml**

Use the SAME docker-compose.yml provided in the Windows section.

TASK 4 — Download PostgreSQL JDBC Driver

Run: **curl -L https://jdbc.postgresql.org/download/postgresql-42.7.3.jar -o jars/postgresql.jar**

Start Docker Services
Run: **docker compose up -d**

Verify Running Containers
Run: **docker ps**

TASK 5 — Open Services

Service	URL
Hadoop HDFS	http://localhost:9870
Spark UI	http://localhost:8080
Jupyter Notebook	http://localhost:8888
Superset	http://localhost:8088

TASK 6 — Download Dataset

Download from: https://www.kaggle.com/datasets/chicago/chicago-food-inspections

Rename: **food_inspection_data.csv**

Move into: **data/source/**

Verify Dataset

Run: head data/source/food_inspection_data.csv

TASK 7 — Create Spark Script

Create: spark/jobs/process.py

Use SAME Python code from the Windows section.

TASK 8 — Run Spark Analytics Job

Run: docker exec -it spark-master /opt/spark/bin/spark-submit /opt/spark/jobs/process.py

Verify Output

Run: **ls data/target/output**

View CSV Results

Run: **cat data/target/output/part-*.csv**

TASK 9 — Verify PostgreSQL Data

Run: docker exec -it postgres psql -U hive -d metastore

Inside PostgreSQL:

SELECT * FROM top_restaurants;

For exit enter: **\q**

TASK 10 — Connect PostgreSQL to Superset

Open: http://localhost:8088
Add database: **postgresql://hive:hive@postgres:5432/metastore**

If there is a connection error then run the following command.

- docker exec -u root -it superset. sh -lc "/app/.venv/bin/python -m ensurepip --upgrade"
- docker exec -u root -it superset sh -lc "/app/.venv/bin/python -m pip install psycopg2-binary"
- docker exec -it superset /app/.venv/bin/python -c "import psycopg2; print('psycopg2 OK')"
- docker restart superset

TASK 11 — Create Dashboard

Build: Charts, Dashboards, Reports
Suggested charts:

- Top Restaurants
- Risk Distribution
- Inspection Failures

TASK 12 — Kafka Streaming

Create Topic

Run: **docker exec kafka kafka-topics --create --topic inspections --bootstrap-server localhost:9092**

Start Producer

Run: **docker exec -it kafka kafka-console-producer --topic inspections --bootstrap-server localhost:9092**

Start Consumer

Run: **docker exec -it kafka kafka-console-consumer --topic inspections --from-beginning --bootstrap-server localhost:9092**

TASK 13 — Spark Streaming Job

Create: spark/jobs/streaming.py

Use SAME Python code from the Windows section.

TASK 14 — Run Streaming Job

Run: docker exec -it spark-master /opt/spark/bin/spark-submit /opt/spark/jobs/streaming.py

TASK 15 — Jupyter Notebook Analytics

Open: http://localhost:8888

Password: mypassword

New → Python 3 (ipykernel)

Create new notebook name: analytics.ipynb

Paste the following code.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
.appName("NotebookAnalytics") \
.getOrCreate()
print("Spark Session Started")
# Load CSV dataset
df = spark.read.csv(
"/home/jovyan/work/food_inspection_data.csv",
header=True,
inferSchema=True
)
# Show schema
df.printSchema()
# Show sample data
```

```
df.show(10)
```

Click: Run

Perform SQL Analytics in Notebook
Create a new notebook cell and Paste the following code.

```
# Create SQL View

df.createOrReplaceTempView("violations")

# SQL Analytics Query

result = spark.sql("""
SELECT
`DBA Name` AS restaurant_name,
COUNT(*) AS total_inspections
FROM violations
GROUP BY `DBA Name`
ORDER BY total_inspections DESC
LIMIT 10
""")
# Display results
result.show()
```

Run the cell.

Create Visualization in Notebook
Create another notebook cell and Paste the following code.

```
import matplotlib.pyplot as plt
pandas_df = result.toPandas()
pandas_df.plot(
x='restaurant_name',
y='total_inspections',
kind='bar'
)
plt.show()
```

Stop Docker Services

Run: docker compose down

Remove Docker Volumes

Run: docker compose down -v